Lekcja 8: Obsługa wyjątków w Pythonie (Wersja rozszerzona)

`#lekcja` `#python` `#wyjątki` `#błędy` `#programowanie-defensywne`

W tej lekcji nauczymy się, jak pisać solidny i niezawodny kod, który potrafi "przetrwać" w obliczu nieoczekiwanych problemów. Zgłębimy mechanizm wyjątków w Pythonie – od ich przechwytywania, przez tworzenie własnych typów błędów, aż po najlepsze praktyki, które sprawią, że nasze programy będą bardziej profesjonalne i odporne na błędy.

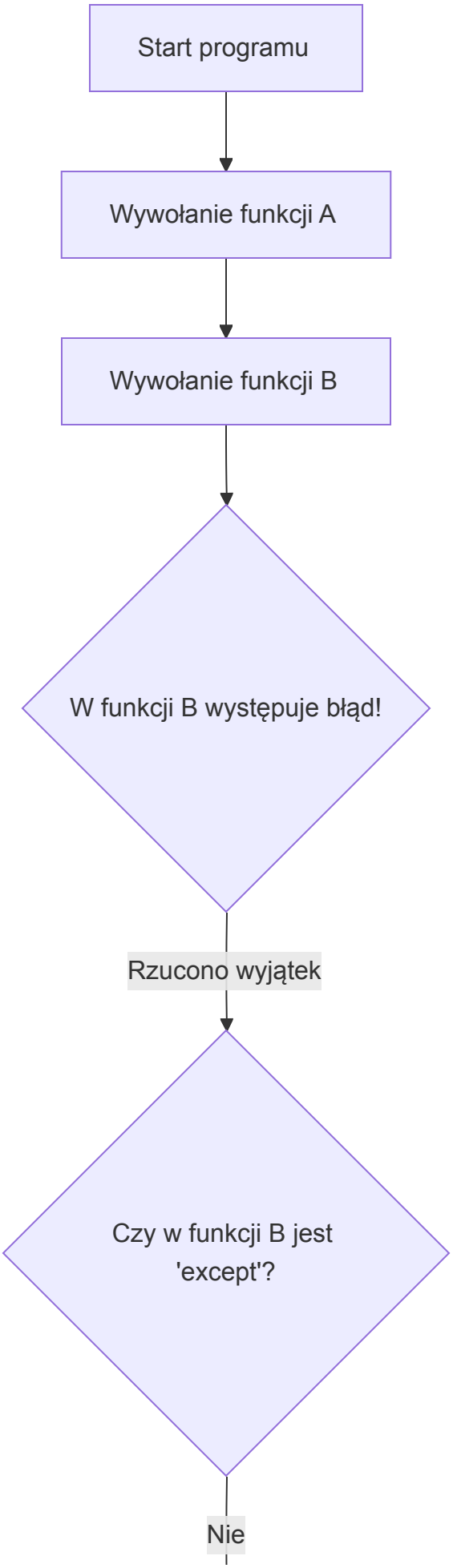
1. Czym są wyjątki i jak działają?

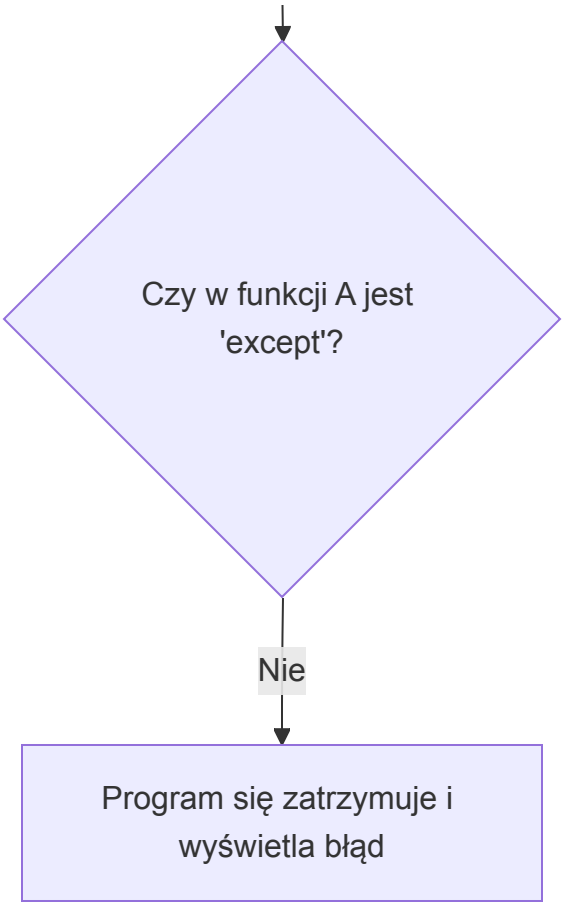
> **Definition**
>
> Wyjątek to obiekt w Pythonie, który reprezentuje błąd lub zdarzenie wyjątkowe, które wystąpiło w trakcie działania programu. Kiedy Python napotyka sytuację, z którą nie może sobie poradzić (np. dzielenie przez zero), przerywa normalny przepływ programu i "rzuca" wyjątkiem (ang. raise an exception).

Analogia: Wyobraź sobie, że jedziesz samochodem (wykonujesz program). Wyjątek jest jak nagła awaria (np. przebita opona). Samochód nie może jechać dalej. Musisz się zatrzymać i podjąć odpowiednie działania (obsłużyć wyjątek), np. zmienić koło. Jeśli nie wiesz, co robić (brak bloku `except`), podróż kończy się w tym miejscu (program się zawiesza).

Co się dzieje, gdy wystąpi wyjątek?

Gdy wyjątek zostaje rzucony, Python "zwija stos wywołań" (ang. *unwinds the call stack*), szukając odpowiedniego "mechanika" – czyli bloku `except`, który potrafi obsłużyć dany typ błędu.


Start programu
Wywołanie funkcji A
Wywołanie funkcji B
W funkcji B występuje błąd!
Rzucono wyjątek
Czy w funkcji B jest 'except'?
Nie



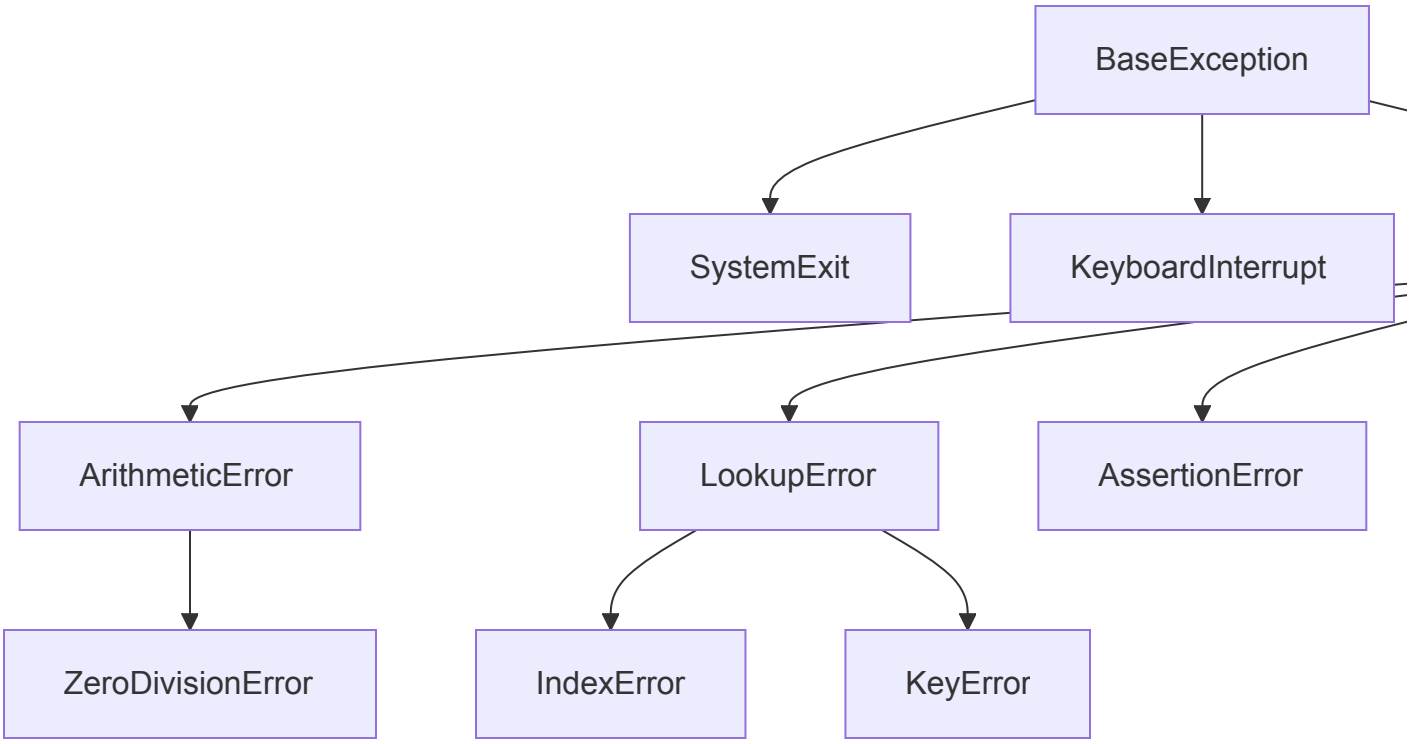
Przykład bez obsługi:

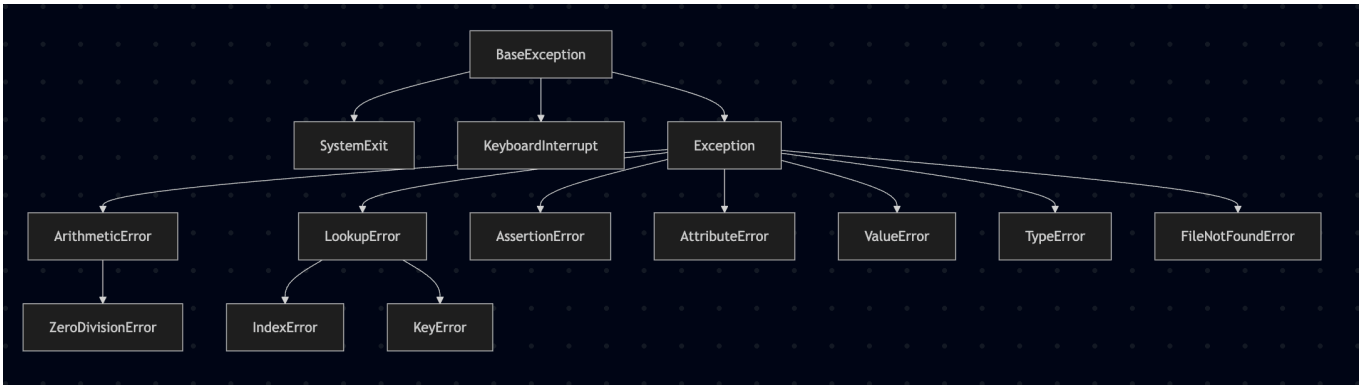
Poniższy kod nigdy nie dotrze do ostatniej linii, ponieważ `ValueError` przerwie jego działanie.

```
print("Zaczynam program...")
liczba = int("nie-jestem-liczbą") # Tutaj wystąpi błąd ValueError
print("Ta linia nigdy się nie wykona.")
```

2. Klasy wyjątków — struktura dziedziczenia

Wszystkie wbudowane wyjątki w Pythonie tworzą hierarchię klas, która dziedziczy z klasy bazowej `BaseException`. W praktyce prawie zawsze pracujemy z wyjątkami dziedziczącymi z `Exception`.





Tip

Zrozumienie tej hierarchii jest ważne. Przechwytując Exception, złapiesz prawie wszystkie standardowe błędy. Przechwytując LookupError, złapiesz zarówno IndexError (błąd indeksu w liście), jak i KeyError (brak klucza w słowniku).

3. Blok `try...except` — jak przechwytywać błędy

To podstawowe narzędzie do obsługi wyjątków.

Składnia:

```
try:
    # Umieść tutaj kod, który może potencjalnie rzucić wyjątek.
    # Nazywamy go "blokiem chronionym".
```

```
except NazwaWyjątku:
    # Ten blok wykona się tylko wtedy, gdy w bloku 'try'
    # wystąpi wyjątek typu 'NazwaWyjątku'.
```

Przykład praktyczny:

```
try:
    wiek_str = input("Podaj swój wiek: ")
    wiek = int(wiek_str)
    print(f"Za rok będziesz mieć {wiek + 1} lat.")
except ValueError:
    print("Błąd: Wprowadzona wartość nie jest poprawną liczbą całkowitą!")
```

4. Obsługa wielu wyjątków

Często jeden fragment kodu może generować różne rodzaje błędów. Możemy je obsługiwać na kilka sposobów.

Kilka bloków `except`

To najlepsze podejście, gdy chcemy inaczej zareagować na każdy typ błędu.

```
try:
    dzielna = 100
    dzielnik = int(input("Podaj liczbę, przez którą chcesz podzielić 100:
"))
    wynik = dzielna / dzielnik
    print(f"Wynik to: {wynik}")
except ValueError:
    print("Musisz podać liczbę!")
except ZeroDivisionError:
    print("Nie wolno dzielić przez zero!")
except Exception as e: # Dobra praktyka: łapanie innych, nieprzewidzianych
błędów
    print(f"Wystąpił nieoczekiwany błąd: {e}")
```

Jeden blok `except` dla wielu typów

Gdy reakcja na różne błędy ma być taka sama.

```
try:
    # ... ten sam kod co wyżej ...
```

```
except (ValueError, ZeroDivisionError) as e:
    print(f"Błąd wprowadzania danych: {e}. Spróbuj ponownie.")
```

> `as e` pozwala przypisać obiekt wyjątku do zmiennej `e`, dzięki czemu możemy wyświetlić jego treść.

5. Bloki `else` i `finally` — dopełnienie obsługi

`else` — wykonuje się, jeśli nie było wyjątku

Blok `else` jest przydatny do umieszczania kodu, który powinien się wykonać tylko wtedy, gdy blok `try` zakończył się sukcesem.

```
try:
    liczba = int(input("Podaj liczbę: "))
except ValueError:
    print("To nie jest liczba.")
else:
    # Ten kod wykona się tylko, jeśli konwersja w 'try' się powiodła
    print(f"Kwadrat Twojej liczby to: {liczba ** 2}")
```

`finally` — wykonuje się zawsze

Blok `finally` jest gwarantowany do wykonania, niezależnie od tego, czy w bloku `try` wystąpił błąd, czy nie, a nawet jeśli funkcja została przerwana przez `return`.

Typowe zastosowanie: zwalnianie zasobów (zamykanie plików, połączeń z bazą danych).

```
plik = None
try:
    plik = open("moje_dane.txt", "r")
    print(plik.read())
except FileNotFoundError:
    print("Plik nie istnieje!")
finally:
    # Ten blok wykona się zawsze, zapewniając zamknięcie pliku
    if plik:
        plik.close()
    print("Zakończono operację na pliku.")
```

6. `raise` — ręczne zgłaszanie wyjątków

Czasami chcemy sami wygenerować błąd, gdy dane wejściowe są niepoprawne z punktu widzenia logiki biznesowej, a nie składni Pythona.

```
def ustaw_wiek(wiek: int):
    if not isinstance(wiek, int):
        raise TypeError("Wiek musi być liczbą całkowitą.")
    if wiek < 0:
        raise ValueError("Wiek nie może być ujemny.")
    print(f"Wiek ustawiony na {wiek}.")

try:
    ustaw_wiek(-5)
except ValueError as e:
    print(f"Błąd logiki: {e}")
```

Można też **"przerzucić" wyjątek dalej** po jego zarejestrowaniu:

```
try:
    wynik = 10 / 0
except ZeroDivisionError as e:
    print(f"Loguję błąd: {e}") # np. zapis do pliku z logami
    raise # Rzucam ten sam wyjątek ponownie, aby wyższy poziom programu mógł
go obsłużyć
```

7. Tworzenie własnych wyjątków

Tworzenie własnych klas wyjątków to dobra praktyka w większych aplikacjach. Sprawia, że kod jest bardziej czytelny i łatwiejszy w zarządzaniu. Własny wyjątek tworzymy, dziedzicząc z klasy `Exception`.

```
# Definiujemy własny typ błędu
class BladWalidacjiHaslaError(Exception):
    """Wyjątek zgłaszany, gdy hasło nie spełnia wymagań."""
    pass

def sprawdz_haslo(haslo: str):
    if len(haslo) < 8:
        raise BladWalidacjiHaslaError("Hasło jest za krótkie (minimum 8
znaków).")
    if not any(char.isdigit() for char in haslo):
        raise BladWalidacjiHaslaError("Hasło musi zawierać co najmniej jedną
cyfrę.")
    print("Hasło jest poprawne.")

try:
    sprawdz_haslo("za-krotkie")
```

```
except BladWalidacjiHaslaError as e:
    print(f"Błąd walidacji: {e}")
```

8. Asercje (`assert`)

Asercja to instrukcja, która sprawdza, czy dany warunek jest prawdziwy. Jeśli nie jest, rzuca wyjątek `AssertionError`.

> ⚠ **Warning**
>
> Asercje służą do debugowania i testowania, a nie do obsługi błędów w działającej aplikacji! Są przeznaczone do sprawdzania wewnętrznych warunków i założeń programisty. Mogą zostać wyłączone podczas uruchamiania Pythona z flagą optymalizacji (-O), więc nie można na nich polegać w logice produkcyjnej.

```
def oblicz_znizke(cena, procent_znizki):
    # Sprawdzamy założenie programisty: zniżka musi być sensowna
    assert 0 <= procent_znizki <= 100, "Procent zniżki musi być między 0 a
100"
    return cena * (procent_znizki / 100)

# To zadziała
print(oblicz_znizke(100, 20))

# To rzuci AssertionError
# print(oblicz_znizke(100, 110))
```

9. Najczęstsze błędy początkujących

- **Brak `try...except`** wokół operacji, które mogą zawieść (np. `int(input())`, otwieranie plików, zapytania sieciowe).
- **Zbyt szeroki `except`**: Łapanie `except Exception:` lub `except:` bez określenia typu "połyka" wszystkie błędy, w tym te, których się nie spodziewaliśmy (np. literówki w nazwach zmiennych), co utrudnia debugowanie.
- **Brak bloku `finally`** przy pracy z zasobami, co może prowadzić do ich "wycieku" (np. niezamknięte pliki).
- **Używanie `assert` do walidacji danych od użytkownika** zamiast `if...raise`.

10. Zadania do samodzielnej pracy

1. **Bezpieczny kalkulator:** Napisz program, który w pętli prosi użytkownika o podanie dwóch liczb i operacji (`+` , `-` , `*` , `/`). Zaimplementuj pełną obsługę błędów `ValueError` (gdy dane nie są liczbami) i `ZeroDivisionError`. Dodaj blok `else` do wyświetlania wyniku i `finally` z komunikatem "Kolejna operacja...".
2. **Walidator wieku:** Stwórz funkcję `rejestruj_uzytkownika(wiek)`, która rzuca własnym, zdefiniowanym przez Ciebie wyjątkiem `WiekNiepoprawnyError`, jeśli wiek jest mniejszy niż 18. Napisz kod, który wywołuje tę funkcję i obsługuje ten wyjątek.
3. **Czytanie pliku:** Napisz funkcję, która próbuje otworzyć i odczytać plik o podanej nazwie. Obsłuż wyjątki `FileNotFoundError` (gdy pliku nie ma) oraz `PermissionError` (gdy nie ma uprawnień do odczytu).
4. **Asercja w funkcji:** Stwórz funkcję `oblicz_srednia(lista_ocen)`, która zwraca średnią z listy. Użyj `assert`, aby upewnić się, że przekazana lista nie jest pusta.
5. **Logowanie błędów:** Zmodyfikuj zadanie 1. tak, aby każdy napotkany wyjątek (wraz z jego treścią) był zapisywany do pliku `log.txt`, a program kontynuował działanie. Użyj bloku `finally`, aby upewnić się, że plik z logami jest zawsze zamykany.
6. **Przerzucanie wyjątku:** Napisz funkcję `przetworz_dane(dane)`, która w bloku `try...except` łapie `KeyError` (np. przy próbie dostępu do nieistniejącego klucza w słowniku), loguje go, a następnie rzuca (`raise`) nowy, własny wyjątek `BladPrzetwarzaniaDanychError` z informacją, którego klucza brakowało.
7. **Bezpieczne pobieranie ze słownika:** Napisz funkcję `pobierz_wartosc(slownik, klucz)`, która bezpiecznie zwraca wartość dla danego klucza. Jeśli klucza nie ma, funkcja nie powinna rzucać błędu, tylko zwracać `None`. Zrób to bez użycia `try...except` (wskazówka: metoda `.get()`). Następnie napisz drugą wersję z użyciem `try...except KeyError`.
8. **Walidacja hasła v2:** Rozbuduj funkcję do walidacji hasła. Powinna ona zwracać listę wszystkich błędów walidacji, zamiast rzucać wyjątkiem po pierwszym napotkanym problemie. Jeśli lista błędów nie jest pusta, rzuć własnym wyjątkiem `BladWalidacjiError`, przekazując do niego tę listę.
9. **Kontekstowy menedżer `with`:** Pokaż, jak instrukcja `with open(...) as f:` upraszcza kod z zadania 3, eliminując potrzebę jawnego używania bloku `finally` do zamykania pliku.
10. Mini-projekt: Sumator liczb z pliku: Napisz program, który:

    a. Pyta użytkownika o nazwę pliku.

    b. Otwiera plik i czyta go linia po linii.

    c. Każdą linię próbuje przekonwertować na liczbę i dodać do sumy.

    d. Ignoruje linie, których nie da się przekonwertować na liczbę (obsługa ValueError).

e. Obsługuje FileNotFoundError, jeśli plik nie istnieje.

f. Na końcu, w bloku finally, wyświetla obliczoną sumę (nawet jeśli wystąpiły błędy po drodze).